Semana 12 de 16 · Reunión N°4 de testeo

Carga semanal automatizada

Del lote de Pluz a la herramienta con una sola orden

DS5045 Ciencia de Datos Computacionales · Pluz Energía y UTEC · Grupo 2

Ricardo Amiel Acuña Villogas	Líder de proyecto y enlace con Pluz
César Eduardo Pajuelo Reyes	Ingeniero de datos
Angello Giovani Soldi Cataño	Analista de integración y exploración
Josué Nehemías Velo Poma	Científico de datos e ingeniero de modelos
Paolo Medrano Terán	Analista de negocio y visualización

Octubre de 2026 · semana del 26 al 30 de octubre

Qué pidió Pluz y qué entregamos

Pedido del 5 de octubre: que la herramienta se refresque con cada lote semanal

El pedido

Pluz envía datos cada semana: interrupciones diarias, poda en formato unificado, guía de subestaciones y ranking SAIDI.

La herramienta tiene que actualizarse sin que nadie rehaga el análisis a mano.

Los datos de Pluz no pueden quedar públicos.

Lo entregado

Un pipeline en Python que convierte cada lote en los datos que dibuja la página.

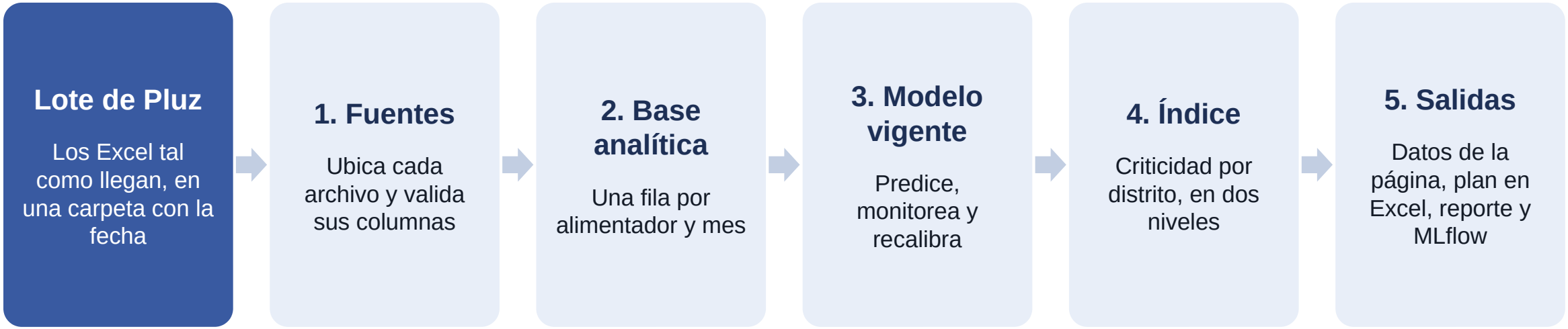
Una sola orden valida, monitorea el modelo, recalcula el índice y deja el plan en Excel.

Los lotes y las salidas técnicas se quedan en el equipo: solo se publican los datos agregados de la página.

Se probó con un lote parcial de agosto: 6.423 interrupciones, sin duplicados

El recorrido del lote: cinco etapas

python pipeline/actualizar.py corre todas, en orden, y se detiene si algo no cuadra



- **Regla de lotes.** Las fuentes históricas se juntan y el lote más reciente manda sobre su periodo; la guía y el ranking son la foto del último lote.
- **Freno de calidad.** Si el modelo queda más de 10 puntos bajo la regla por historial, la corrida no publica.

Las órdenes del pipeline

Subcomandos sin opciones, pensados para quien no programa

`$ python pipeline/actualizar.py validar`	Solo revisa que los lotes cumplan el contrato de datos
`$ python pipeline/actualizar.py`	Corrida completa: valida, monitorea, calcula y escribe los datos
`$ python pipeline/actualizar.py ver`	Abre la página en el navegador, servida desde el equipo
`$ python pipeline/actualizar.py subir`	Publica los datos de la página, previa confirmación
`$ python pipeline/actualizar.py semanal`	Lo de cada lunes: recibe el lote, corre todo y copia la página
`$ pipeline/programar/actualizar_semanal.bat`	Tarea programada de Windows; en Linux, el .sh con cron

Contrato de datos: si el lote no cumple, la corrida se detiene

Cada fuente declara su nombre de archivo, su hoja y sus columnas en pipeline/config.py

- **Se reconoce por el nombre.** Sin distinguir mayúsculas ni tildes: no hace falta renombrar los Excel.
- **Se valida antes de calcular.** Si falta una columna o una hoja, dice cuál y en qué lote.
- **Histórico o foto.** Interrupciones y poda se acumulan entre lotes; la guía y el ranking valen solo en su última versión.
- **Un cambio de formato de Pluz** se ajusta en un solo lugar, sin tocar el resto del pipeline.

```
$ python pipeline/actualizar.py validar
1. Fuentes: localizar y validar los lotes
  interrupciones: 6339 filas
  poda: 8545 filas
  ranking_saidi: 674 filas
  guia_sed: 14318 filas
  diccionario_causas: 40 filas
  postes_mt: 0 filas
  AVISO: 12 alimentadores con eventos de vegetación no figuran en la guía de subestaciones
  Interrupciones del 2024-01-01 al 2026-08-07
Los lotes cumplen el contrato de datos.
```

Salida real en modo demostración

El modelo no se reentrena cada semana

Un modelo vigente estable que se vigila con cada lote

Monitorea

En los meses nuevos que ya tienen resultado compara su orden con la regla por historial y su probabilidad con lo observado

Recalibra

Si la probabilidad se aleja de lo observado ajusta la escala sin cambiar el orden del ranking

Recomienda

Si el orden cae avisa que conviene reentrenar; se reentrena solo con la orden reentrenar

Cada corrida queda registrada en MLflow: el monitoreo semanal y el benchmark de modelos

Cómo probarlo sin tocar la operación: modo demostración

Repetición con los datos reales: agosto de 2026 llega en tres Excel semanales, uno por uno

1	`$ python3 pipeline/demo/preparar_demo.py`	Arma la demostración, con datos hasta julio
2	`$ export PLUZ_DEMO=$PWD/pipeline/datos/demo`	Desde aquí todo trabaja dentro de la demostración
3	`$ python3 pipeline/demo/preparar_demo.py siguiente`	Pluz entrega la semana: el Excel llega a la carpeta de entrada
4	`$ ./pipeline/programar/actualizar_semanal.sh`	La corrida del lunes: recibe, actualiza y copia
5	`$ python3 pipeline/actualizar.py ver`	Revisar la herramienta con el corte nuevo
6	`$ python3 pipeline/demo/preparar_demo.py borrar`	Borra todo: la operación real nunca se tocó

Lo que sigue: que no haga falta la consola

La carga funciona, pero exige escribir órdenes y no muestra qué trae el archivo antes de usarlo

- **Semana 13:** la misma carga desde el navegador, con vista previa del archivo, comparación con lo ya cargado, confirmación y deshacer.
- **Para Pluz:** que el ingeniero de mantenimiento cargue la semana sin depender del equipo de datos.

Herramienta	**https://ricardoamiel.github.io/CDC-Pluz-Poda/**
Benchmark	**https://ricardoamiel.github.io/CDC-Pluz-Poda/benchmark/**
Carga semanal	**https://ricardoamiel.github.io/CDC-Pluz-Poda/cargar/**